

Clasificación de imágenes con una CNN

Inteligencia artificial · Licenciatura en Sistemas

1. Problema y alcance

Se implementó y entrenó una red neuronal convolucional para clasificar imágenes RGB de 32 × 32 píxeles en las diez categorías de CIFAR-10. La variable de salida es una clase mutuamente excluyente; la red produce probabilidades mediante softmax. El objetivo es evaluar generalización con validación por grupos, estudiar empíricamente sesgo y varianza y entregar un modelo Keras reutilizable. No se aborda la parte de redes recurrentes.

2. Datos y prevención de filtraciones

CIFAR-10 contiene originalmente 50.000 imágenes de entrenamiento y 10.000 de prueba [1]. Antes de muestrear, se detectaron imágenes con píxeles idénticos mediante SHA-256: se descartaron 0 de entrenamiento y 0 de prueba. Se conserva la primera aparición y se da prioridad a entrenamiento, sin usar etiquetas para decidir qué eliminar. Esto evita duplicados exactos entre particiones, pero no detecta imágenes casi idénticas ni demuestra independencia semántica.

Propiedad	Ejecución documentada
Desarrollo utilizado	10,000 imágenes; selección estratificada dentro de cada lote
Prueba retenida	10,000 imágenes; consultada tras fijar arquitectura y épocas
Grupos	Los cinco archivos originales data_batch_1 a data_batch_5
Clases	airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
Semilla y presupuesto	Semilla base 42; máximo 10 épocas; minibatch 128
Escala de entrada	RGB 0-255; Rescaling(1/255) incorporado al modelo

GroupKFold deja un lote completo como validación y utiliza los otros cuatro para aprender [4]. Cada imagen de desarrollo se valida una vez. Los lotes son grupos de almacenamiento, no identificadores de sujeto, captura o procedencia: esta separación no garantiza independencia por fuente. La estratificación se aplica al muestreo dentro de cada lote; GroupKFold no asegura proporciones de clases idénticas entre folds.

El muestreo reducido permite reproducir el experimento con recursos moderados. No se eliminaron imágenes de prueba: se preservan las 10.000 imágenes oficiales. Se entrena con 10,000 imágenes de desarrollo; comparar con trabajos que usan 50.000 exige considerar ese presupuesto diferente. El manifiesto guarda índices de selección y hashes de los píxeles para auditar la ejecución.

3. Arquitectura y fundamentos

Una capa convolucional comparte filtros en todas las posiciones, reduciendo parámetros y favoreciendo patrones locales. En Keras, Conv2D implementa correlación cruzada: $y[i,j,f] = b[f] + \sum(u,v,c) W[u,v,c,f] x[i \cdot s + u \cdot p, j \cdot s + v \cdot p, c]$. La convolución matemática estricta invierte el filtro; en aprendizaje sus pesos libres hacen habitual llamar convolución a esta operación [3].

Por dimensión espacial, $n_{out} = \text{floor}((n + 2p - k)/s) + 1$. Con $k = 3$, $s = 1$ y $\text{padding} = \text{'same'}$, $p = 1$ conserva ancho y alto. Cada MaxPooling2D usa ventana 2×2 y stride 2: reduce cada dimensión a la mitad, conserva el máximo local y no agrega pesos.

Capa	Salida	Parámetros
Entrada + Rescaling	$32 \times 32 \times 3$	0
Conv2D 16, 3×3 + ReLU	$32 \times 32 \times 16$	448
MaxPool 2×2	$16 \times 16 \times 16$	0
Conv2D 32, 3×3 + ReLU	$16 \times 16 \times 32$	4.640
MaxPool 2×2	$8 \times 8 \times 32$	0
Conv2D 64, 3×3 + ReLU	$8 \times 8 \times 64$	18.496
MaxPool 2×2 + Flatten	$4 \times 4 \times 64 \rightarrow 1.024$	0
Dense 64 + ReLU + Dropout(0,3)	64	65.600
Dense 10 + Softmax	10	650
Total entrenable		89.834

Cada convolución tiene $(k^2 \cdot C_{entrada} + 1) \cdot C_{salida}$ parámetros; la densa tiene $(N_{entrada} + 1) \cdot N_{salida}$. $\text{ReLU}(z) = \max(0, z)$ posee derivada 1 para z positivo y 0 para z negativo: evita la saturación positiva de sigmoid, aunque puede producir neuronas inactivas. He normal usa varianza $2/\text{fan_in}$ y ayuda a conservar la escala de activaciones y gradientes. No garantiza por sí sola un entrenamiento estable.

4. Optimización y selección

Softmax transforma logits en probabilidades: $p_k = \exp(z_k) / \sum_j \exp(z_j)$. La función objetivo es $J = -(1/N) \sum_i \log p(y_i | x_i) + \lambda \sum_{\text{kernel}} \|W\|^2$, con $\lambda = 0,0001$.

Se minimiza entropía cruzada categórica dispersa más penalización L2 de 0,0001 en los kernels convolucionales y la densa oculta. Adam usa tasa 0,001 y clipnorm=1: recorta la norma del gradiente de cada variable, no su norma global conjunta. Dropout 0,3 regulariza la capa oculta solo al entrenar. No se aplica aumento de datos.

En cada fold, EarlyStopping observa val_loss (paciencia 3) y restaura los mejores pesos. La mediana de las mejores épocas fue 10; con ese número fijo se reentrenó sobre todo desarrollo. La arquitectura se fijó antes del test; no hubo búsqueda de hiperparámetros ni ajuste de épocas usando sus resultados.

5. Validación cruzada y sesgo-varianza

Fold / grupo validado	Acc. train	Acc. val	F1 macro val	Brecha (pp)
1 / 5	57.90%	52.15%	0.5156	5.75
2 / 4	59.13%	53.35%	0.5360	5.78
3 / 3	63.54%	56.20%	0.5533	7.34
4 / 2	61.31%	53.20%	0.5307	8.11
5 / 1	59.96%	51.20%	0.5074	8.76

Accuracy de validación: media 53.22%, desviación estándar muestral 1.88 puntos porcentuales (pp). La brecha train-validación media es 7.15 pp (rango 5.75 a 8.76). Se calcula con los pesos restaurados, en inferencia y sin dropout en ambas particiones; no se resta el accuracy de entrenamiento registrado durante actualizaciones del modelo.

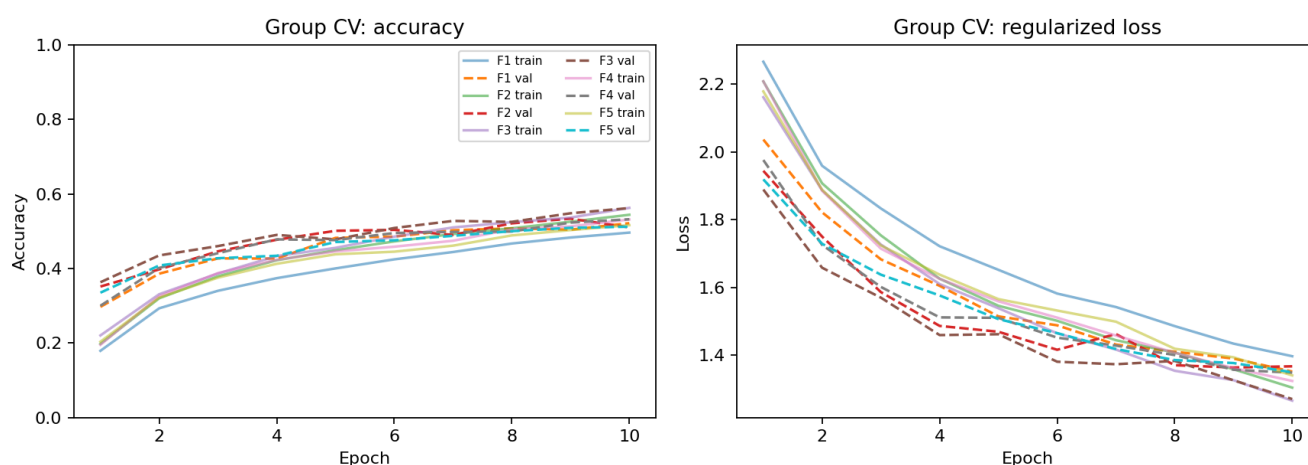


Figura 1. Línea continua: entrenamiento; discontinua: validación. La pérdida de las curvas incluye L2, a diferencia de la entropía cruzada predictiva informada en las tablas.

Diagnóstico: mismo holdout	n train	Acc. train	Acc. val
Lineal: promedio 4×4 + softmax	8000	34.54%	33.65%
CNN, mitad de datos	4000	51.38%	47.75%
CNN, datos completos del fold	8000	57.90%	52.15%

La CNN supera al modelo lineal en 18.50 pp de validación en ese holdout. Al pasar de la mitad a todos los datos del fold, la variación es +4.40 pp. Ambos diagnósticos usan la semilla del fold 1, el mismo holdout, máximo de épocas y criterio de parada; las épocas efectivas pueden diferir. El lineal cambia capacidad y representación. Con la mitad de datos hay menos actualizaciones por época: la comparación mezcla cantidad de datos y pasos de optimización.

El error medio es 39.63% en entrenamiento y 46.78% en validación. La mayor tasa de error fuera del entrenamiento es compatible con sobreajuste. En 4/5 folds, la mejor época coincide con el máximo de 10: el presupuesto finito no demuestra convergencia. La optimización incompleta y el sobreajuste por muestra limitada pueden coexistir. La dispersión entre folds mezcla lote e inicialización; no descompone matemáticamente sesgo y varianza. Una corrida por condición no permite atribución causal: hacen falta semillas repetidas, más tamaños de muestra y grupos de procedencia reales.

6. Evaluación final y análisis de errores

Partición / agregación	Accuracy	F1 macro	Entropía cruzada
Validación fuera de fold (OOF)	53.22%	0.5291	1.3034
Desarrollo, modelo final	59.91%	0.5878	1.1179
Prueba retenida, modelo final	53.42%	0.5218	1.2870

Accuracy mide aciertos totales; F1 macro promedia por clase la media armónica de precisión y recall y pondera por igual las diez categorías. La entropía cruzada penaliza especialmente errores con alta confianza. OOF concatena predicciones de cinco modelos; prueba corresponde a un único modelo reentrenado.

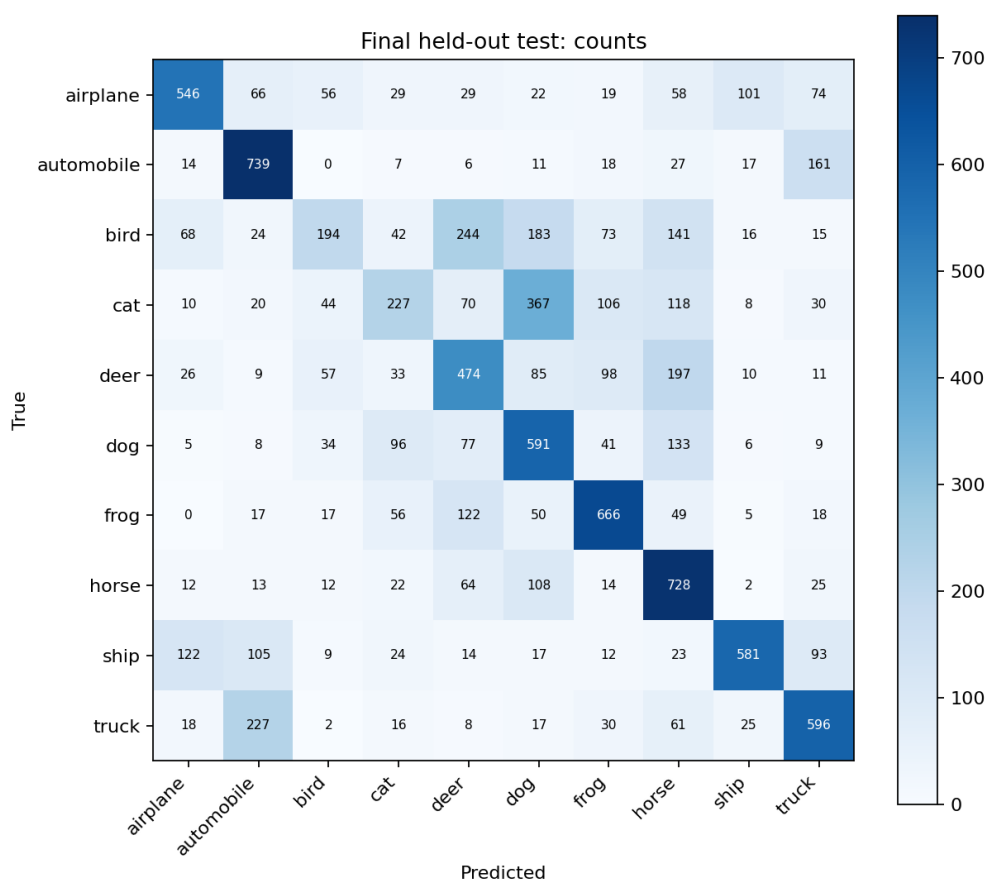


Figura 2. Matriz de confusión en prueba: filas = clase real, columnas = predicción; los valores son cantidades absolutas, no porcentajes.

Las confusiones direccionales más frecuentes son cat → dog (367 casos); bird → deer (244 casos); truck → automobile (227 casos). Son errores observados; la matriz por sí sola no identifica qué rasgo visual causó cada uno.

La clase con mayor F1 es automobile (0.6634); la menor es bird (0.2723). El desempeño desigual aconseja revisar ejemplos por categoría antes de extender el modelo a imágenes ajenas a CIFAR-10. No demuestra que haya aprendido exclusivamente rasgos del objeto ni descarta atajos ligados a fondos u otros artefactos.

7. Resultados por clase y robustez

Clase	Precisión	Recall	F1	Soporte
airplane	0.665	0.546	0.600	1000
automobile	0.602	0.739	0.663	1000
bird	0.456	0.194	0.272	1000
cat	0.411	0.227	0.293	1000
deer	0.428	0.474	0.450	1000
dog	0.407	0.591	0.482	1000
frog	0.618	0.666	0.641	1000
horse	0.474	0.728	0.574	1000
ship	0.754	0.581	0.656	1000
truck	0.578	0.596	0.587	1000

Al trasladar cada imagen de prueba 2 píxeles hacia abajo y a la derecha, la accuracy es 39.62% (-13.80 pp respecto del original), y F1 macro 0.3686. Se rellenan con ceros el borde superior e izquierdo y se recortan dos filas y columnas del extremo opuesto. Por ello, el efecto combina traslación, pérdida de contenido y cambio de borde; no es una prueba aislada de invariancia.

Compartir filtros favorece equivariancia local antes de bordes y submuestreo. El max pooling puede aportar tolerancia parcial a pequeños desplazamientos, pero stride 2 cambia el alineamiento de ventanas y no garantiza equivariancia exacta. Flatten y capas densas también dependen de la posición. Esta prueba no permite atribuir la respuesta solo al pooling.

8. Reproducibilidad y conclusión

Entorno registrado: Python 3.10.11, TensorFlow 2.16.2, NumPy 1.26.4. Se fijan semillas y operaciones deterministas. El modelo `results/model.keras` incorpora normalización y acepta lotes RGB 32 × 32 en escala 0-255. Tras guardarlo y recargarlo, la diferencia máxima de probabilidades sobre 32 imágenes fue 0.

La ejecución alcanza 53.42% de accuracy en la prueba depurada. Las curvas, brechas, referencia lineal y reducción de datos sustentan un diagnóstico empírico limitado por el presupuesto y una sola semilla por condición. El repositorio incluye entrenamiento, inferencia, manifiesto, métricas y predicciones auditables. Para mejorar, corresponde evaluar aumento de datos y mayor presupuesto exclusivamente en desarrollo, reservando una nueva prueba si las decisiones usan estos resultados finales.

Referencias

- [1] Krizhevsky, A. CIFAR-10: datos y descripción. cs.toronto.edu/~kriz/cifar.html
- [2] Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. [Informe técnico](#).
- [3] Keras. Conv2D layer. [Documentación oficial de Conv2D](#).
- [4] scikit-learn. GroupKFold. [Documentación oficial de GroupKFold](#).